Tutorial Sheet — TD 1 / Correction Guidelines

→ Solution – Exercise 1

Using the standard growth hierarchy (each function is asymptotically dominated by the next):

$\log n \prec \sqrt{n} \prec n \prec n \cdot \log n \prec n^2 \prec n^3 \prec 2^n \prec n!$

Justification sketch:

- **log n** grows slower than any positive power of **n** ($\sqrt{n} = n^{0.5}$).
- Polynomial powers **n, n·log n, n²**, **n³** are ordered by their degree (the **log n** factor grows slower than any **ε>0** power of **n,** so **n·log n** sits strictly between **n** and **n²**).
- Exponential 2^n eventually dominates every polynomial n^k (by L'Hôpital or the ratio test on $n^k / 2^n \to 0$).
- Finally **n!** dominates 2^n, e.g. via Stirling's approximation $n! \approx \sqrt{(2\pi n)}(n/e)^n$, which grows faster than any fixed base raised to **n**.

→ Solution – Exercise 2

For a fixed **i,** the inner loop runs for **j = i, i+1, …, n,** i.e. **(n − i + 1)** times. Summing over **i**:

$\sum_{i=1}^{n}(n - i + 1) = n + (n-1) + \ldots + 1 = n(n+1)/2$

Hence the total number of operations is **n(n+1)/2**, which is **Θ(n²)**.

→ Model Answer (Corrigé Type) [Exercise 3]

The outer loop runs exactly **n** times, contributing a factor of **n**.

For each execution of the outer loop, the inner while-loop doubles **j** starting from **1** until **j ≥ n**. The number of doublings needed is $\lceil \log_2 n \rceil$, so the inner loop runs **Θ(log n)** times, independently of **i**.

Since the inner loop's cost does not depend on the outer loop's index **i,** the total cost is the product of the two factors: **n × Θ(log n) = Θ(n log n)**

Common mistakes to avoid:

- Assuming the two loops' costs simply add **(n + log n)** instead of multiplying, because the inner loop is nested inside the outer one.
- Forgetting that the inner loop's iteration count depends on **n,** not on the changing value of **i,** and mistakenly writing **Θ(n · log i)**.